Міністерство освіти і науки України

Національний технічний університет України

“Київський політехнічний інститут імені Ігоря Сікорського”

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Лабораторна робота №1

Методи і технології паралельного програмування

Тема: Засоби створення та керування потоками в паралельних мультипоточних програмах

Виконав:

студент групи ІП-51мн

Панченко С. В.

Перевірив:

Лесик В. О.

Київ 2026

ЗМІСТ

1 Мета 4
2 Завдання 5
2.1 Основне завдання 5
2.2 Реалізація прикладів задач трьох типів 5
2.2.1 CPU-bound задачі 5
2.2.2 Memory-bound задачі 5
2.2.3 IO-bound задачі 5
2.3 Дослідження продуктивності 5
2.4 Вимоги до звіту 5
3 Виконання 7
3.1 CPU-bound 7
3.1.1 Обчислення числа π методом Монте-Карло з різною кількістю ітерацій 7
3.1.1.1 Опис алгоритму 7
3.1.1.2 Реалізація 7
3.1.1.3 Дослідження продуктивності 7
3.1.2 Факторизація великих чисел та обчислення простих чисел в заданому діапазоні 9
3.1.2.1 Опис алгоритму 9
3.1.2.2 Реалізація 9
3.1.2.3 Дослідження продуктивності 10
3.2 Memory-bound 11
3.2.1 Опис алгоритму 11
3.2.2 Реалізація 11
3.2.3 Дослідження продуктивності 12
3.3 IO-bound 13
3.3.1 Опис алгоритму 13
3.3.2 Реалізація 13
3.3.3 Дослідження продуктивності 14
4 Контрольні питання 16
4.1 Які були застосовані фреймворки та які їх особливості та відмінності від інших фреймворків? 16
4.2 Поясніть різницю між CPU-bound, Memory-bound та I/O-bound задачами. 16
4.3 Що таке закон Амдала і як він впливає на прискорення паралельних програм? 17
4.4 Чому I/O-bound задачі можуть ефективно використовувати більше потоків, ніж кількість ядер процесора? 17

4.5 Що таке overhead при створенні потоків і як він впливає на продуктивність? 17
4.6 Поясніть концепцію пулу потоків (thread pool) та його переваги. 17
4.7 Які проблеми можуть виникати при роботі з Memory-bound задачами в багатопоточному середовищі? 17
5 Висновок 19
A Лістинг коду 20
A.1 count_words.cpp 20
A.2 gen_dir.cpp 25
A.3 matrix_transpose.cpp 29
A.4 pi_monte.cpp 36
A.5 prime_slice.cpp 42

1 МЕТА

Ознайомитися з основними засобами створення та керування потоками в паралельних мультипоточних програмах, а також дослідити ефективність паралельної обробки для різних типів задач.

2 ЗАВДАННЯ

2.1 Основне завдання

Реалізувати послідовну та паралельну мультипоточну обробку незалежних задач.

2.2 Реалізація прикладів задач трьох типів

2.2.1 CPU-bound задачі

Вимоги: Складні обчислення з невеликим обсягом даних.

Реалізувати наступний ряд задач:

- Обчислення числа π методом Монте-Карло з різною кількістю ітерацій
- Факторизація великих чисел
- Обчислення простих чисел в заданому діапазоні

2.2.2 Memory-bound задачі

Вимоги: Інтенсивна робота з даними в оперативній пам’яті

Реалізувати наступну задачу:

- Транспонувати матрицю великої розмірності (10000 x 10000)

2.2.3 IO-bound задачі

Опис: Робота з даними на диску

Реалізувати наступну задачу:

- Рекурсивно відкрити директорію та підрахувати загальну кількість слів у всіх текстових файлах; текстові файли та директорію згенерувати випадковим чином, додати значну кількість файлів – до 1000, в допустимих межах.

2.3 Дослідження продуктивності

Виміряти та проаналізувати залежність часу виконання від кількості потоків для кожного типу задач.

2.4 Вимоги до звіту

- Титулка
- Тема, Мета, Завдання
- Короткий опис обраного підходу до рішення задачі, опис критичних моментів рішення.
- Код проєкту, обґрунтування обрання фреймворку, опис відмінностей та переваг.
- Рисунки із демонстрацією виконання програми.

- Рисунки із демонстрацією прискорення та різними значеннями параметрів.
- Відповіді на контрольні запитання.
- Висновки

3 ВИКОНАННЯ

3.1 CPU-bound

3.1.1 Обчислення числа π методом Монте-Карло з різною кількістю ітерацій

3.1.1.1 Опис алгоритму

Алгоритм Монте-Карло для знаходження числа π базується на геометричній імовірності. Суть методу полягає у генерації великої кількості випадкових точок із координатами (x,y) у межах квадрата зі стороною 2 (де x та y належать проміжку від −1 до 1). Далі для кожної точки перевіряється умова її потрапляння у вписане коло одиничного радіуса: $x^2 + y^2 \leq 1$. Оскільки площа кола дорівнює πr^2, а площа квадрата — $4r^2$, відношення кількості точок, що потрапили в коло, до загальної кількості згенерованих точок наближається до $\frac{\pi}{4}$.

3.1.1.2 Реалізація

Для досягнення максимальної продуктивності у програмі було відмовлено від стандартних генераторів випадкових чисел C++, оскільки вони можуть створювати надмірні накладні витрати. Натомість реалізовано швидкий генератор Xoroshiro128PP. Кожен потік створює власний екземпляр генератора та ініціалізує його унікальним seed-значенням.

Загальна кількість точок для генерації (points_count) рівномірно розподіляється між доступними потоками (std::jthread). Якщо кількість точок не ділиться націло на кількість потоків, залишок розподіляється по одній додатковій ітерації на перші потоки.

Для синхронізації етапів виконання використовується примітив pthread_barrier_t. Алгоритм працює наступним чином:

1. Головний потік ініціалізує бар'єр і запускає робочі потоки.
2. Робочі потоки виконують свою частину обчислень (викликають функцію count_points) і записують локальний результат у спільний масив points_circle_counts.
3. Після завершення своїх обчислень усі потоки зупиняються на бар'єрі (pthread_barrier_wait).
4. Як тільки всі потоки досягають бар'єра, головний потік за допомогою std::accumulate підсумовує локальні результати, обчислює підсумкове значення π та перевіряє його коректність (MY_ASSERT).

3.1.1.3 Дослідження продуктивності

Для аналізу продуктивності програма проводить бенчмаркінг: обчислення виконуються для кількості точок від 100 млн до 1 млрд. Спочатку заміряється час

послідовного виконання, а потім — паралельного (з кількістю потоків від 2 до 12). Для нівелювання похибок кожен тест виконується по 10 разів.

На рисунках 3.1, 3.2 можна побачити результати продуктивності.

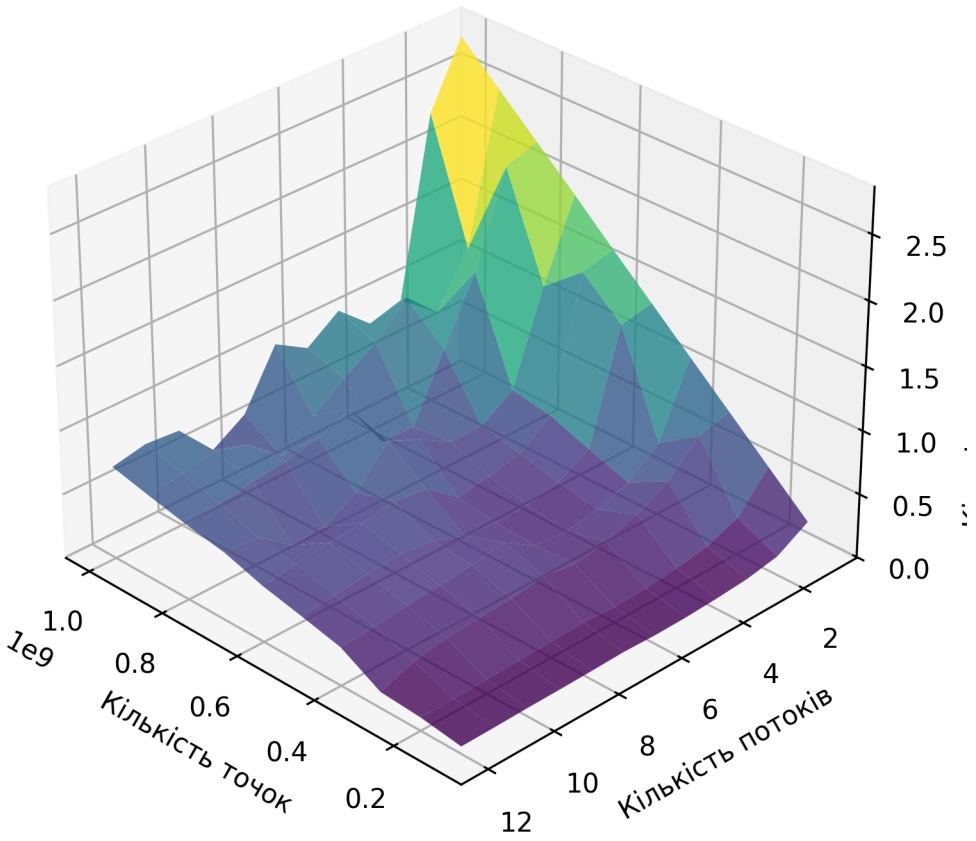

Рисунок 3.1 — Залежність часу від кількості точок та потоків

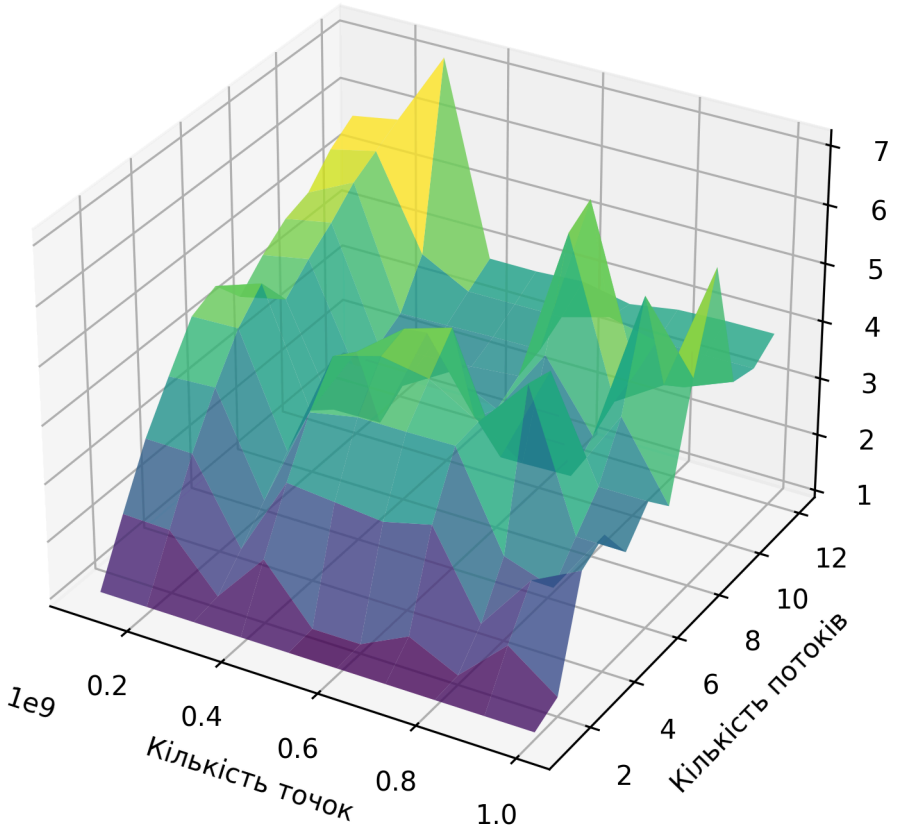

Рисунок 3.2 — Залежність прискорення від кількості точок та потоків

Аналіз результатів показує чітку лінійну залежність часу виконання від кількості точок у послідовному режимі. Паралельна обробка демонструє високу ефективність: два потоки зменшують час виконання майже вдвічі, що підтверджує мінімальні накладні витрати завдяки незалежним генераторам випадкових чисел. Зі збільшенням кількості потоків (до 12 доступних ядер) загальний час обчислень продовжує скорочуватися, проте графік прискорення поступово відхиляється від ідеального. Це сповільнення приросту продуктивності є закономірним і пояснюється накладними витратами ОС на перемикання контексту та очікуванням на фінальному бар'єрі синхронізації.

3.1.2 Факторизація великих чисел та обчислення простих чисел в заданому діапазоні

3.1.2.1 Опис алгоритму

Задача полягає у пошуку всіх простих чисел у великому числовому діапазоні [mmin, mmax). Для кожного числа виконується математична перевірка на простоту.

3.1.2.2 Реалізація

Для того, щоб уникнути впливу підсистеми вводу-виводу (I/O bottleneck) на чистоту експерименту, результати перевірки кожного числа (булеве

значення) направляються у системний псевдопристрій /dev/null за допомогою низькорівневого системного виклику write. Це гарантує, що задача залишається суто обчислювальною (CPU-bound).

Загальний діапазон [mmin, mmax) розділяється на рівні піддіапазони, які паралельно обробляються потоками std::jthread. Синхронізація виконання, як і в попередній задачі, забезпечується примітивом pthread_barrier_t, що гарантує точний замір часу виконання кожної ітерації.

3.1.2.3 Дослідження продуктивності

Аналіз даних показує суттєве зменшення часу роботи багатопоточної версії. Наприклад, послідовна обробка найважчого діапазону (16.7 млн – 33.5 млн) займає близько 19.5 секунд, тоді як використання двох потоків скорочує час до 10.7 секунд (прискорення майже 1.8x). При подальшому масштабуванні швидкість продовжує зростати. Невелика втрата ідеальної лінійності прискорення пояснюється дисбалансом навантаження (load imbalance): потоки, що обробляють кінець діапазону, працюють з більшими числами, а отже, витрачають більше процесорного часу на їх перевірку порівняно з потоками, що обробляють початок діапазону.

На рисунках 3.3, 3.4 можна побачити результати продуктивності.

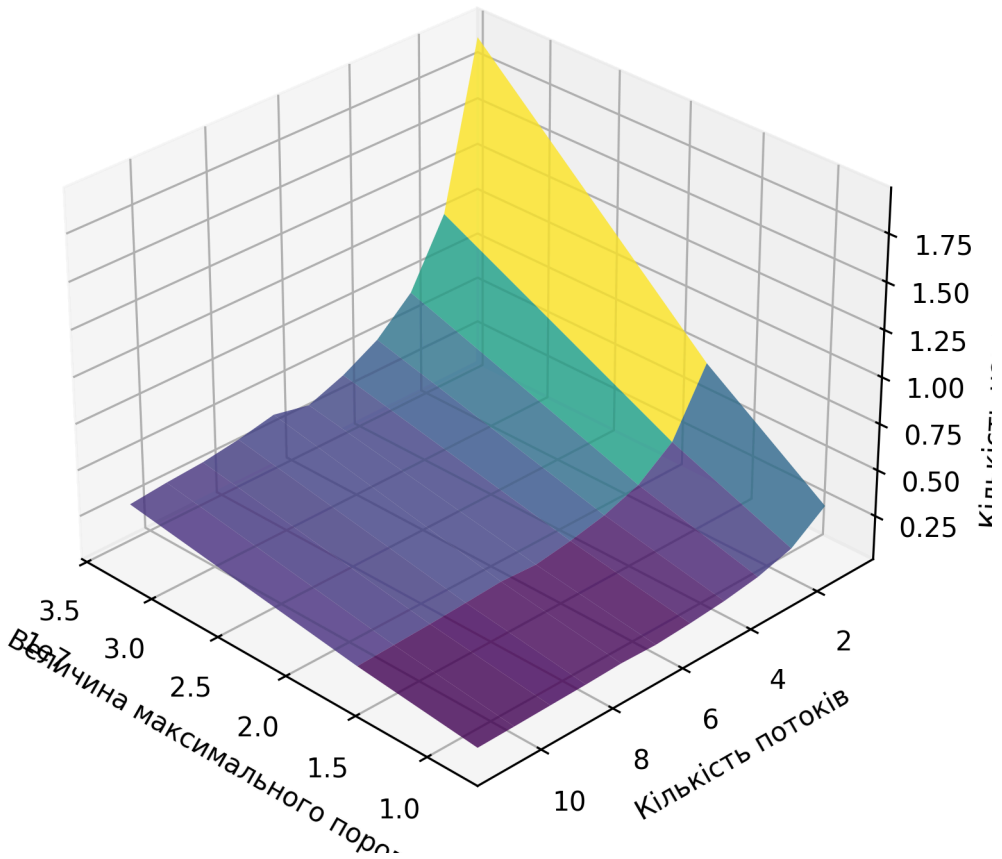

Рисунок 3.3 — Залежність часу від величини порогу та потоків

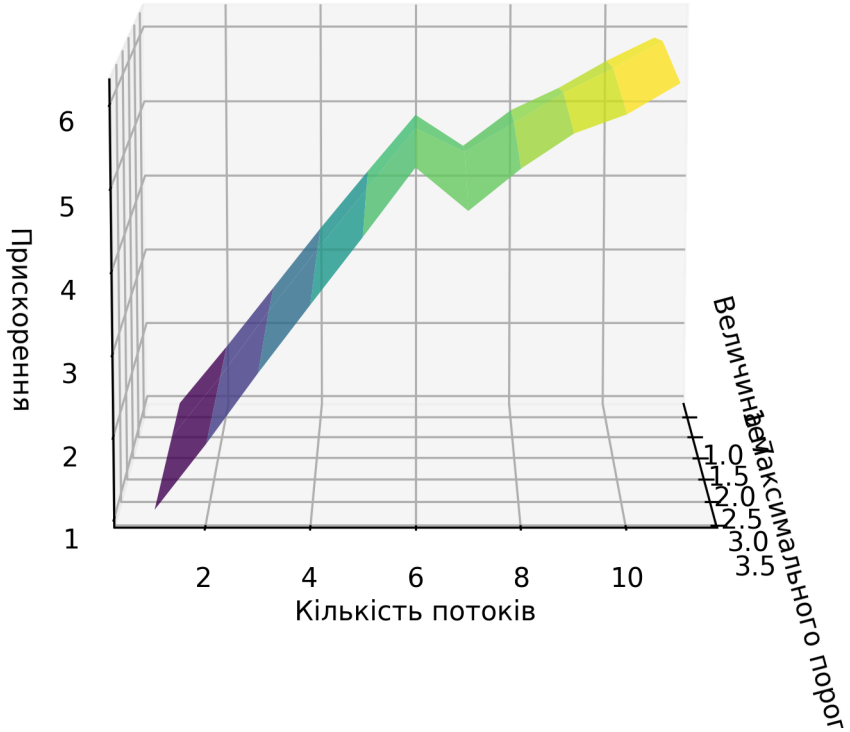

Рисунок 3.4 — Залежність прискорення від величини порогу та потоків

3.2 Memory-bound

3.2.1 Опис алгоритму

Транспонування матриці є класичною Memory-bound задачею. Хоча сама операція обміну елементів є тривіальною, головною проблемою стає шаблон доступу до пам’яті. При читанні матриці по рядках і записі по стовпцях виникає величезна кількість промахів кешу (cache misses), оскільки дані завантажуються в кеш-лінії процесора, але використовується лише один елемент з лінії, після чого лінія витісняється. Це призводить до того, що швидкість роботи обмежується не процесором, а пропускною здатністю шини оперативної пам’яті.

3.2.2 Реалізація

Матриця розміром 10000×10000 зберігається у вигляді одновимірного масиву std::vector для забезпечення неперервності пам’яті. Ключовою оптимізацією алгоритму є застосування блокового (tiled) транспонування. Матриця розбивається на менші підматриці (блоки) розміром 64x64 елементи (BLOCK_SIZE = 64). Такий розмір підібрано таким чином, щоб два робочі блоки (той, що читається, і той, куди йде запис) повністю поміщалися у швидку кеш-пам’ять L1/L2 процесора. Функції transpose_block_diag та transpose_block_swap виконують транспонування локально всередині цих блоків.

Розпаралелювання відбувається на рівні блоків. Замість того, щоб ділити матрицю на великі суцільні шматки, потоки отримують ітерації з певним кроком (i += threads_count * BLOCK_SIZE), що забезпечує більш рівномірне балансування навантаження. Для синхронізації та вимірювання часу також використовується pthread_barrier_t.

3.2.3 Дослідження продуктивності

Результати бенчмаркінгу яскраво демонструють специфіку Memory-bound задач. Звичайне (наївне) розпаралелювання часто може навіть погіршити продуктивність через конкуренцію потоків за шину пам'яті (memory bus contention) та явище хибного розділення (false sharing). Проте поєднання блокової оптимізації з багатопоточністю дає відмінний результат: час послідовного виконання становить близько 1.39 секунди, тоді як використання кількох потоків з блоковим підходом зменшує цей час до 0.3–0.4 секунди. Подальше збільшення кількості потоків не дає лінійного ідеального прискорення, оскільки алгоритм швидко впирається у фізичну межу пропускної здатності (bandwidth) оперативної пам'яті комп'ютера.

На рисунках 3.5, 3.6 можна побачити результати продуктивності.

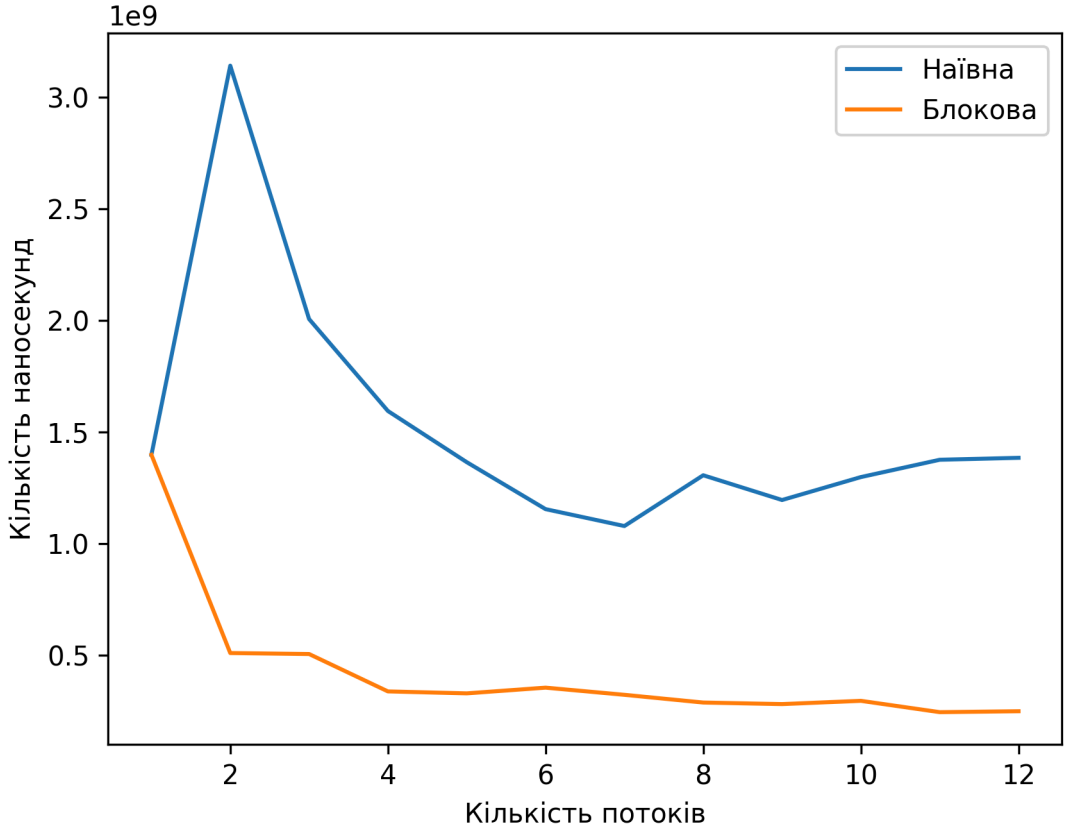

Рисунок 3.5 — Залежність часу від кількості потоків

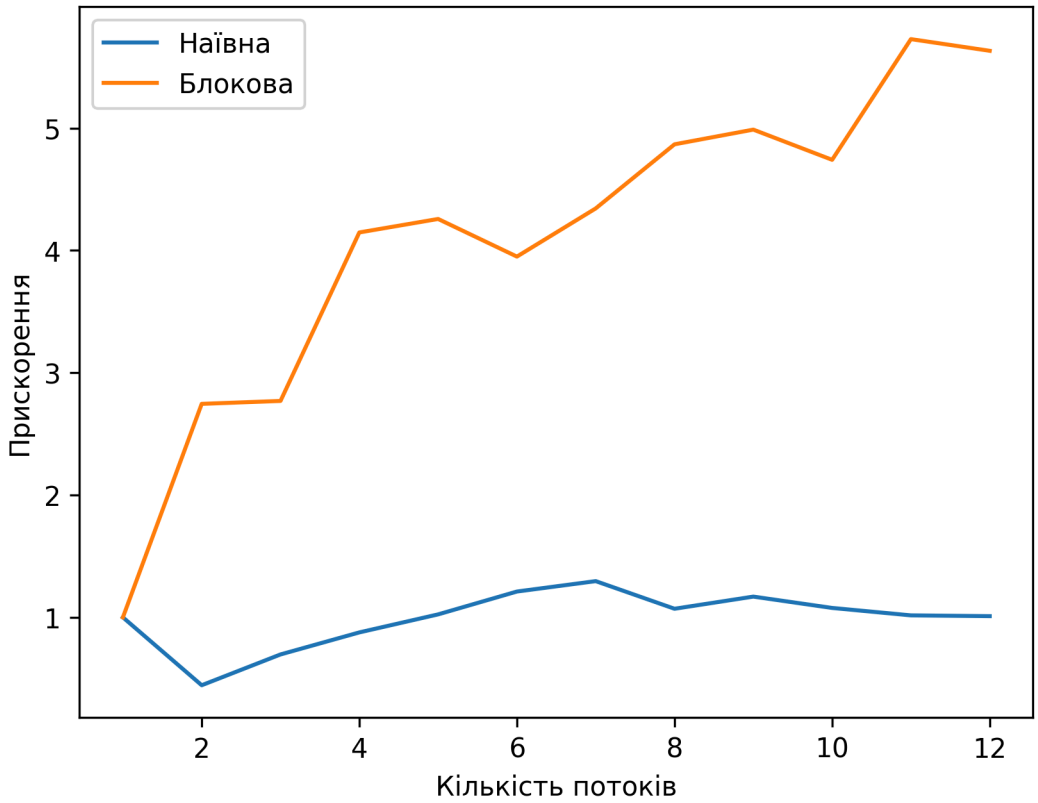

Рисунок 3.6 — Залежність прискорення від кількості потоків

3.3 IO-bound

3.3.1 Опис алгоритму

Підрахунок слів у великій кількості файлів є типовою IO-bound (обмеженою вводом-виводом) задачею. Основний час виконання програми витрачається не на арифметичні обчислення (пошук пробілів чи символів нового рядка), а на взаємодію з файловою системою: системні виклики для відкриття файлів, читання даних з диска (або дискового кешу ОС) та закриття дескрипторів. Для тестування було написано допоміжну програму gen_dir.cpp, яка генерує тестову директорію з 10 000 текстових файлів однакового розміру.

3.3.2 Реалізація

Для досягнення максимальної швидкодії замість стандартних потоків C++ (std::ifstream) було використано системний виклик mmap (Memory-Mapped Files). Цей підхід дозволяє відобразити вміст файлу безпосередньо у віртуальний адресний простір процесу. Це усуває необхідність проміжного копіювання даних з простору ядра (kernel space) у простір користувача (user space) і дозволяє операційній системі максимально ефективно використовувати механізми сторінкового кешування (page cache).

Обробка розпаралелюється на рівні файлів. Спочатку головний потік рекурсивно обходить директорію (за допомогою std::filesystem) і збирає шляхи до всіх файлів у єдиний вектор. Далі робочі потоки (std::jthread) проходять по

цьому масиву за принципом циклічного розподілу (Round-Robin): кожен потік бере файл, індекс якого зміщується на загальну кількість потоків (file_index += threads_count). Це забезпечує рівномірне балансування навантаження між потоками. Для точного вимірювання часу всі потоки стартують одночасно за допомогою pthread_barrier_t.

3.3.3 Дослідження продуктивності

У послідовному режимі обробка 10 000 файлів займає близько 0.99 секунди. Використання багатопоточності спочатку дає гарний приріст: 2 потоки зменшують час до 0.53 с, а 4 потоки — до 0.29 с.Проте, починаючи з 4-5 потоків, масштабування різко зупиняється, і час виконання виходить на “плато” (для 12 потоків час становить 0.27 с, що майже не відрізняється від результату 5 потоків). Це пояснюється тим, що програма впирається в апаратне обмеження — максимальну пропускну здатність (Bandwidth) та кількість операцій вводу-виводу за секунду (IOPS) дискової підсистеми або у пропускну здатність шини пам’яті (якщо файли вже закешовані в RAM). Коли цей ліміт досягнуто, додавання нових обчислювальних потоків не дає жодного ефекту, оскільки процесорні ядра просто простоюють в очікуванні даних від операційної системи.

На рисунках 3.7, 3.8 можна побачити результати продуктивності.

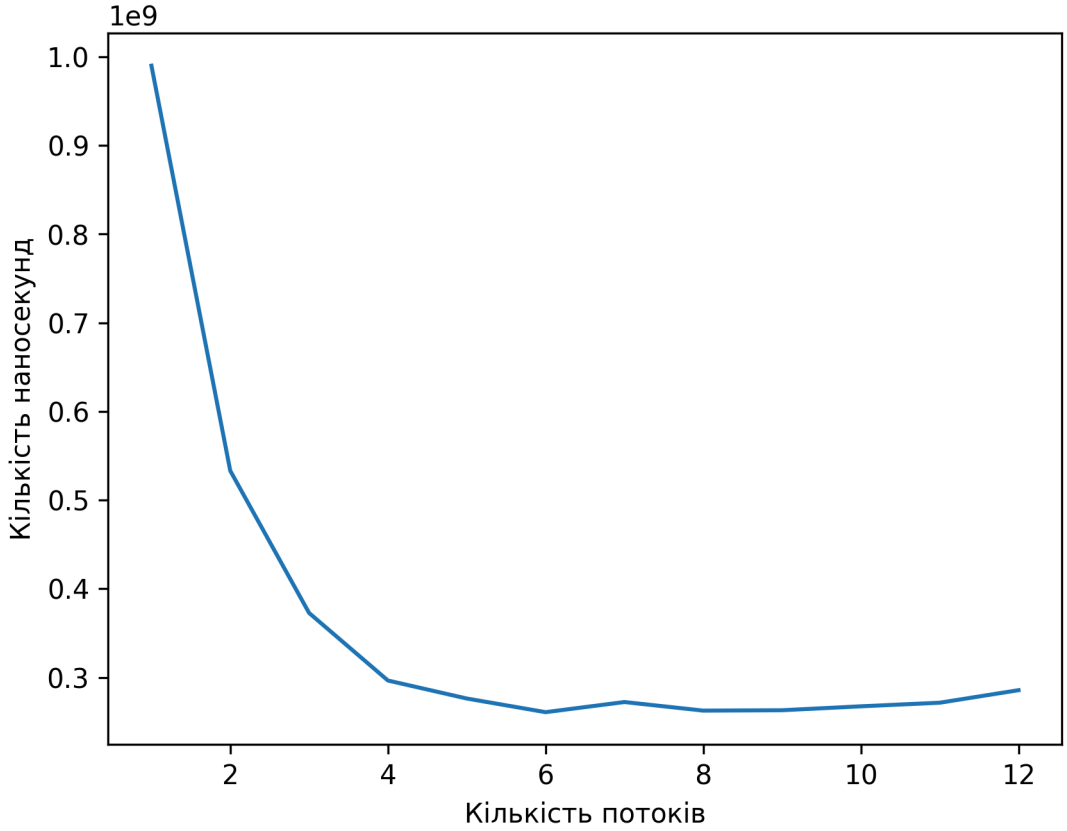

Рисунок 3.7 — Залежність часу від кількості потоків

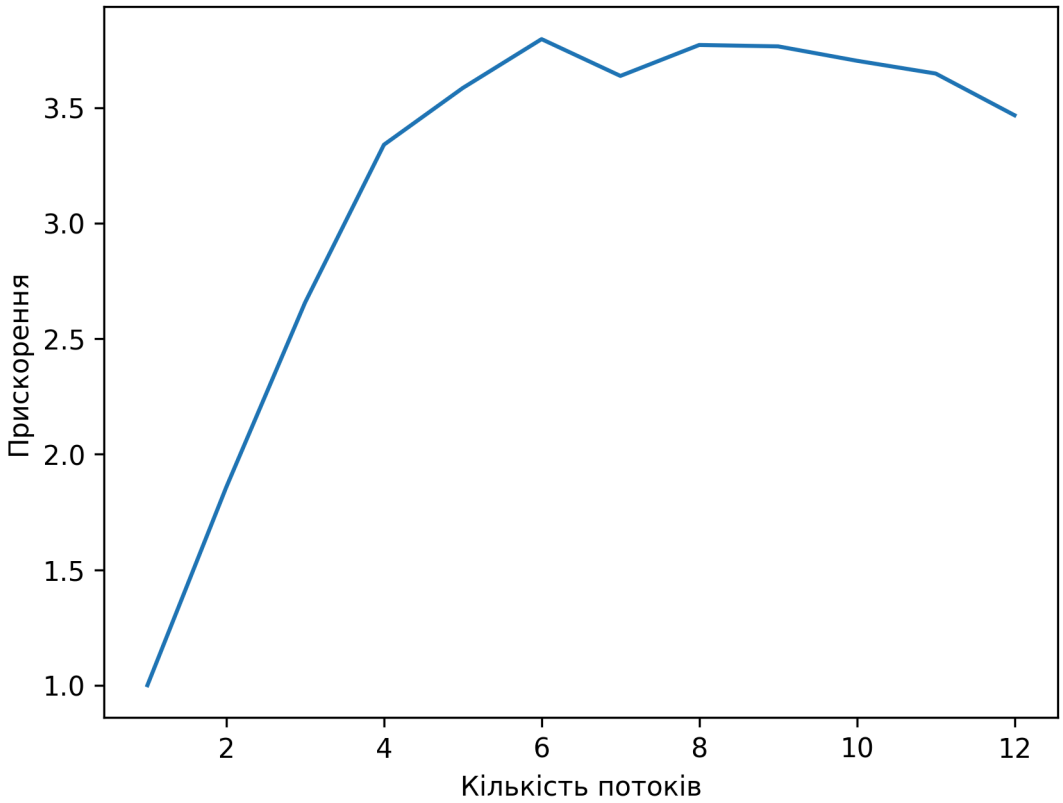

Рисунок 3.8 — Залежність прискорення від кількості потоків

4 КОНТРОЛЬНІ ПИТАННЯ

4.1 Які були застосовані фреймворки та які їх особливості та відмінності від інших фреймворків?

Використав std::thread для керування потоками та pthread_barrier_t для синхронізації етапів обчислень. Для C++ існує багато фреймовків для паралельних обчислень. Обмежимося згадкою boost та OpenMP.

OpenMP та Boost представляють два принципово різних підходи до паралелізму в C++. OpenMP — це стандарт, що базується на директивах компілятора, призначений насамперед для високоефективних обчислень (HPC) та розпаралелювання циклів. Його головна перевага — простота: ви додаєте спеціальний рядок (прагму) над важким циклом, і компілятор сам бере на себе створення потоків, розподіл ітерацій та синхронізацію. Це ідеальний інструмент для математичних розрахунків, обробки великих масивів даних та алгоритмів, де однакові операції виконуються над багатьма елементами одночасно. Однак OpenMP обмежений моделлю Fork-Join і погано підходить для розробки асинхронних систем або складних мережевих додатків.

На противагу йому, Boost (зокрема модулі Thread та Asio) — це набір бібліотек, що надають низькорівневий контроль над потоками та асинхронністю. Якщо OpenMP фокусується на паралелізмі даних, то Boost — на паралелізмі задач та ефективному керуванні ресурсами. Boost.Thread дозволяє вручну створювати потоки та керувати м’ютексами, а Boost.Asio є стандартом де-факто для асинхронного вводу-виводу, дозволяючи програмі обробляти тисячі з’єднань одночасно, не створюючи окремий потік для кожного. Boost набагато гнучкіший і потужніший для створення системного ПЗ, серверів або GUI, але він вимагає написання значно більшої кількості коду та глибшого розуміння архітектури програми. Підсумовуючи: OpenMP обирають для швидкості розрахунків, а Boost — для архітектурної гнучкості та асинхронної взаємодії.

4.2 Поясніть різницю між CPU-bound, Memory-bound та I/O-bound задачами.

CPU-bound це обчислювально складні операції, як-от криптографія або рендеринг відео, де прискорення можливе лише через збільшення тактової частоти або кількості ядер. Memory-bound задачі залежать від пропускної здатності або затримки оперативної пам’яті; процесор часто простоює, чекаючи на дані з RAM, що характерно для обробки великих масивів даних. I/O-bound задачі обмежені швидкістю підсистеми вводу-виводу (диск, мережа, термінал); більшість часу потік перебуває в стані очікування зовнішньої відповіді, а не виконує активні обчислення.

4.3 Що таке закон Амдала і як він впливає на прискорення паралельних програм?

Закон Амдала визначає теоретичне максимальне прискорення програми при її паралельному виконанні залежно від частки коду, який неможливо розпаралелити (послідовна частина). Згідно з формулою $\frac{1}{(1-p)+\frac{p}{n}}$, p де — частка коду, що паралелиться, а n — кількість ядер, навіть при нескінченній кількості процесорів загальне прискорення буде обмежене послідовним компонентом. Це означає, що оптимізація алгоритму (зменшення) часто є важливішою для продуктивності, ніж просте додавання нових ядер.

4.4 Чому I/O-bound задачі можуть ефективно використовувати більше потоків, ніж кількість ядер процесора?

I/O-bound задачі ефективно масштабуються за межі кількості ядер, оскільки потік, що очікує на дані з мережі або диска, переходить у стан блокування і звільняє процесор. Поки один потік “спить” в очікуванні відповіді від бази даних, ОС може переключити ядро на виконання іншого потоку. Це дозволяє підтримувати високий рівень паралелізму та обробляти тисячі запитів одночасно, оскільки реальне навантаження на обчислювальні потужності при цьому залишається мінімальним.

4.5 Що таке overhead при створенні потоків і як він впливає на продуктивність?

Overhead (накладні витрати) — це ресурси процесора та пам’яті, які витрачаються на службові операції замість виконання корисної роботи. При створенні потоку ОС має виділити стек (зазвичай 1 МБ), ініціалізувати структури в ядрі та згодом витрачати час на перемикання контексту (context switching) між ними. Якщо потоків занадто багато, система починає витрачати більше часу на керування ними, ніж на самі обчислення, що призводить до деградації продуктивності та надмірного споживання RAM.

4.6 Поясніть концепцію пулу потоків (thread pool) та його переваги.

Пул потоків — це патерн проектування, при якому створюється фіксована або динамічна кількість робочих потоків заздалегідь, які очікують на завдання в черзі. Основні переваги: економія ресурсів, оскільки потоки використовуються повторно замість постійного створення/видалення; контроль навантаження, що запобігає падінню системи через занадто велику кількість активних потоків; та швидкість, бо завдання починає виконуватися миттєво без затримки на ініціалізацію нового системного ресурсу.

4.7 Які проблеми можуть виникати при роботі з Memory-bound задачами в багатопоточному середовищі?

У Memory-bound задачах головною проблемою є конкуренція за шину пам’яті та кеш-пам’ять. Коли багато потоків одночасно намагаються читати або записувати великі обсяги даних, вони створюють “вузьке місце” в каналі доступу до RAM. Також виникає проблема false sharing (ложного розділення), коли потоки

на різних ядрах оновлюють дані, що знаходяться в одному кеш-рядку, змушуючи процесор постійно інвалідувати та синхронізувати кеші, що різко сповільнює роботу системи навіть за наявності вільних ядер.

5 ВИСНОВОК

У ході лабораторної роботи досліджено ефективність багатопоточної обробки.

CPU-bound: Демонструють майже лінійне прискорення до досягнення ліміту фізичних ядер. Основний фактор успіху — рівномірне балансування навантаження між потоками.

Memory-bound: Швидкодія жорстко обмежується пропускною здатністю оперативної пам’яті. Реального прискорення вдалося досягти лише завдяки оптимізації доступу до кешу (блокове транспонування).

IO-bound: Багатопоточність дає приріст лише на малих кількостях потоків. Далі продуктивність швидко досягає плато через обмеження дискової підсистеми (IOPS), і додавання нових потоків втрачає сенс.

Додаток А
Лістинг коду

A.1 count_words.cpp

```
#include <concepts>
#include <string_view>
#include <vector>
#include <chrono>
#include <thread>

#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <string.h>
#include <pthread.h>

#include <filesystem>
#include <dirent.h>
#include <sys/mman.h>

#define MY_STRINGIFY_IMPL(...) #__VA_ARGS__
#define MY_S(...) MY_STRINGIFY_IMPL(__VA_ARGS__)
#define MY_CONCAT_INTERNAL(a, b) a ## b
#define MY_CONCAT(a, b) MY_CONCAT_INTERNAL(a, b)
#define MY_FOR_RANGE(type, name, mmin, mmax) for(type name = (mmin);
name < (mmax); name++)
#define MY_FOR_RANGE_ZERO(name, mmax) for(auto name =
static_cast<typename std::remove_cv<decltype(mmax)>::type>(0); name <
(mmax); name++)

#define MY_ASSERT(expr) \
    do {\
        if(!(expr)) {\
            fprintf(stderr, "%s:%d: %s: assertion `%s` failed.\n",
__FILE__, __LINE__, __FUNCTION__, #expr);\
            abort();\
        }\
    } while(0)

#define MY_ASSERT_EXT(expr, ...) \
    do {\
        if(!(expr)) {\
            fprintf(stderr, "%s:%d: %s: assertion `%s` failed.\n",
__FILE__, __LINE__, __FUNCTION__, #expr);\
            fprintf(stderr, __VA_ARGS__);\
            abort();\
        }\
```

```
    } while(0)

#define MY_ASSERT_NOT_LESS_ZERO(expr) \
    do {\
        if((expr) < 0) {\
            fprintf(stderr, "%s:%d: %s: errno: %d: strerror: %s:
assertion `%s' failed.\n", __FILE__, __LINE__, __FUNCTION__, errno,
strerror(errno), #expr);\
            abort();\
        }\
    } while(0)

#define MY_LOG_DEBUG_SINGLE(fmt) \
    do {\
        fprintf(stderr, "[%s:%d] [%s] [%ld] [" fmt "]\n", __FILE__,
__LINE__, __FUNCTION__, time(nullptr));\
    } while(0)

#define MY_LOG_DEBUG(fmt, ...) \
    do {\
        fprintf(stderr, "[%s:%d] [%s] [%ld] [" fmt "]\n", __FILE__,
__LINE__, __FUNCTION__, time(nullptr) __VA_OPT__(,) __VA_ARGS__);\
    } while(0)

#define MY_PRINT_EXPR_IMPL(expr, format)\
    do {\
        MY_LOG_DEBUG("`"#expr"`: `" format "`", (expr));\
    } while(0)


template <typename F>
struct defer_t {
    F f;
    ~defer_t() { f(); }
};
#define defer(code) const defer_t MY_CONCAT(_defer_, __LINE__){[&]()
{ code; }}

#define MY_ARRAY_COUNT(arr) (sizeof(arr) / sizeof(arr[0]))

#define MY_CHECKED_WRITE(fd, s) do { const auto
MY_CONCAT(_my_checked_write_, __LINE__) = write(fd, s.data(),
s.length()); MY_ASSERT_NOT_LESS_ZERO(MY_CONCAT(_my_checked_write_,
__LINE__)); MY_ASSERT(static_cast<size_t>(MY_CONCAT(_my_checked_write_,
__LINE__)) == s.length()); } while(0)

#define MY_PTHREAD_BARRIER_WAIT(barrier)\
    do {\
```

```
        switch(pthread_barrier_wait(barrier)) {\
            case 0:\
            case PTHREAD_BARRIER_SERIAL_THREAD: {\
                break;\
            }\
            default: {\
                MY_ASSERT(false);\
                break;\
            }\
        }\
    } while(0)

template<typename T>
static std::chrono::system_clock::duration timeit(T&& func) {
    const auto start = std::chrono::system_clock::now();
    func();
    return std::chrono::system_clock::now() - start;
}

static constexpr size_t cpus_count = 12;
static constexpr size_t samples_count = 10;

template<auto MMIN, auto MMAX, class F>
static constexpr void static_for(F&& f)
    requires std::same_as<decltype(MMIN), decltype(MMAX)> &&
std::integral<decltype(MMIN)>
{
    if constexpr (MMIN < MMAX) {
        std::forward<F>(f).template operator()<MMIN>();
        static_for<MMIN + 1, MMAX>(std::forward<F>(f));
    }
}

int main() {
    MY_ASSERT(sysconf(_SC_NPROCESSORS_ONLN) ==
static_cast<long>(cpus_count));

    const auto current_file = std::filesystem::path(__FILE__);
    const auto project_dir = current_file.parent_path();
    const auto build_dir = project_dir / "build";
    const auto files_dir = build_dir / "test_dir";

    DIR *const files_dir_ptr = opendir(files_dir.c_str());
    MY_ASSERT(files_dir_ptr);
    defer(MY_ASSERT_NOT_LESS_ZERO(closedir(files_dir_ptr)));
    const int files_dir_fd = dirfd(files_dir_ptr);
    defer(MY_ASSERT_NOT_LESS_ZERO(files_dir_fd));
```

```
    std::vector<std::filesystem::path> file_paths;
    for(const auto& entry :
std::filesystem::directory_iterator{files_dir}) {
        if(entry.is_regular_file()) {
            file_paths.emplace_back(entry.path().filename());
        }
    }

    const int dev_null_fd = open("/dev/null", O_WRONLY);
    MY_ASSERT_NOT_LESS_ZERO(dev_null_fd);
    defer(MY_ASSERT_NOT_LESS_ZERO(close(dev_null_fd)));

    const auto count_words = [files_dir_fd, dev_null_fd](const
std::basic_string_view<char> name) {
        const int fd = openat(files_dir_fd, name.data(), O_RDONLY);
        MY_ASSERT_NOT_LESS_ZERO(fd);
        defer(MY_ASSERT_NOT_LESS_ZERO(close(fd)));
        struct stat st;
        MY_ASSERT_NOT_LESS_ZERO(fstat(fd, &st));
        char *const ptr = reinterpret_cast<char*>(mmap(nullptr,
static_cast<size_t>(st.st_size), PROT_READ, MAP_SHARED, fd, 0));
        MY_ASSERT(ptr != MAP_FAILED);
        defer(MY_ASSERT_NOT_LESS_ZERO(munmap(ptr,
static_cast<size_t>(st.st_size))));
        size_t word_count = 0;
        char *const it_end = ptr + st.st_size;
        char *it = ptr;
        while(true) {
            it = std::find(it, it_end, '\n');
            if(it == it_end) {
                break;
            }
            it++;
            word_count++;
        }
        write(dev_null_fd, &word_count, sizeof(word_count));
    };

    const auto warm_up = [&file_paths, &count_words]() {
        MY_LOG_DEBUG("start warm_up");
        const auto dur = timeit([&file_paths, &count_words]() {
            std::array<std::jthread, cpus_count> threads{};
            for(auto& t : threads) {
                t = std::jthread([&file_paths, &count_words]() {
                    for(const auto& f : file_paths) {
                        count_words(f.native());
                    }
                });
```

```
            }
        });
        const auto dur_casted =
std::chrono::duration_cast<std::chrono::duration<double,
std::milli>>(dur);
        MY_LOG_DEBUG("end warm_up, took %lf ms", dur_casted.count());
    };
    warm_up();
    {
        MY_LOG_DEBUG("start single");
        const auto filename = current_file.stem().filename().native() +
"_single.csv";
        const auto fd = open(filename.c_str(), O_RDWR | O_CREAT |
O_TRUNC);
        MY_ASSERT_NOT_LESS_ZERO(fd);
        defer(MY_ASSERT_NOT_LESS_ZERO(close(fd)));
        MY_ASSERT_NOT_LESS_ZERO(fchmod(fd, 0666));
        constexpr std::string_view header = "nanoseconds\n";
        MY_CHECKED_WRITE(fd, header);
        MY_FOR_RANGE_ZERO(sample_index, samples_count) {
            const auto dur = timeit([&]() {
                for(const auto& f : file_paths) {
                    count_words(f.native());
                }
            });
            const auto line = std::to_string(dur.count()) + "\n";
            MY_CHECKED_WRITE(fd, line);
            printf("%s", line.data());
        }
        MY_LOG_DEBUG("end single");
    }
    warm_up();
    {
        MY_LOG_DEBUG("start multi");

        const auto filename = current_file.stem().filename().native() +
"_multi.csv";
        const int fd = open(filename.c_str(), O_RDWR | O_CREAT |
O_TRUNC);
        MY_ASSERT_NOT_LESS_ZERO(fd);
        defer(MY_ASSERT_NOT_LESS_ZERO(close(fd)));
        MY_ASSERT_NOT_LESS_ZERO(fchmod(fd, 0666));
        constexpr std::string_view header =
"threads_count,nanoseconds\n";
        MY_CHECKED_WRITE(fd, header);

        MY_FOR_RANGE(size_t, threads_count, 2, cpus_count + 1) {
            MY_FOR_RANGE_ZERO(sample_index, samples_count) {
```

```
                pthread_barrier_t barrier{};
                MY_ASSERT_NOT_LESS_ZERO(pthread_barrier_init(&barrier,
nullptr, static_cast<uint32_t>(threads_count + 1)));

defer(MY_ASSERT_NOT_LESS_ZERO(pthread_barrier_destroy(&barrier)));

                std::vector<std::jthread> threads(threads_count);
                MY_FOR_RANGE_ZERO(thread_index, threads_count) {
                    threads[thread_index] = std::jthread(
                        [&barrier, thread_index, threads_count,
&file_paths, &count_words]() {
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                            for(size_t file_index = thread_index;
file_index < file_paths.size() - 1; file_index += threads_count) {

count_words(file_paths[file_index].native());
                            }
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                        }
                    );
                }
                MY_PTHREAD_BARRIER_WAIT(&barrier);
                const auto dur = timeit(
                    [&barrier]() {
                        MY_PTHREAD_BARRIER_WAIT(&barrier);
                        MY_PTHREAD_BARRIER_WAIT(&barrier);
                    }
                );
                const auto line = std::to_string(threads_count) + "," +
std::to_string(dur.count()) + "\n";
                MY_CHECKED_WRITE(fd, line);
                printf("%s", line.data());
            }
        }
        MY_LOG_DEBUG("end multi");
    }
    return 0;
}
```

A.2 gen_dir.cpp

```
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <string.h>
#include <pthread.h>
#include <sys/mman.h>
```

```
#include <filesystem>

#define MY_STRINGIFY_IMPL(...) #__VA_ARGS__
#define MY_S(...) MY_STRINGIFY_IMPL(__VA_ARGS__)
#define MY_CONCAT_INTERNAL(a, b) a ## b
#define MY_CONCAT(a, b) MY_CONCAT_INTERNAL(a, b)
#define MY_FOR_RANGE(type, name, mmin, mmax) for(type name = (mmin);
name < (mmax); name++)
#define MY_FOR_RANGE_ZERO(name, mmax) for(auto name =
static_cast<typename std::remove_cv<decltype(mmax)>::type>(0); name <
(mmax); name++)

#define MY_ASSERT(expr) \
    do {\
        if(!(expr)) {\
            fprintf(stderr, "%s:%d: %s: assertion `%s` failed.\n",
__FILE__, __LINE__, __FUNCTION__, #expr);\
            abort();\
        }\
    } while(0)

#define MY_ASSERT_EXT(expr, ...) \
    do {\
        if(!(expr)) {\
            fprintf(stderr, "%s:%d: %s: assertion `%s` failed.\n",
__FILE__, __LINE__, __FUNCTION__, #expr);\
            fprintf(stderr, __VA_ARGS__);\
            abort();\
        }\
    } while(0)

#define MY_ASSERT_NOT_LESS_ZERO(expr) \
    do {\
        if((expr) < 0) {\
            fprintf(stderr, "%s:%d: %s: errno: %d: strerror: %s:
assertion `%s' failed.\n", __FILE__, __LINE__, __FUNCTION__, errno,
strerror(errno), #expr);\
            abort();\
        }\
    } while(0)

#define MY_LOG_DEBUG_SINGLE(fmt) \
    do {\
        fprintf(stderr, "[%s:%d] [%s] [%ld] [" fmt "]\n", __FILE__,
__LINE__, __FUNCTION__, time(nullptr));\
    } while(0)

#define MY_LOG_DEBUG(fmt, ...) \
```

```
    do {\
        fprintf(stderr, "[%s:%d] [%s] [%ld] [" fmt "]\n", __FILE__,
__LINE__, __FUNCTION__, time(nullptr) __VA_OPT__(,) __VA_ARGS__);\
    } while(0)

#define MY_PRINT_EXPR_IMPL(expr, format)\
    do {\
        MY_LOG_DEBUG("`"#expr"`: `" format "`", (expr));\
    } while(0)


template <typename F>
struct defer_t {
    F f;
    ~defer_t() { f(); }
};
#define defer(code) const defer_t MY_CONCAT(_defer_, __LINE__){[&]()
{ code; }}

#define MY_ARRAY_COUNT(arr) (sizeof(arr) / sizeof(arr[0]))

#define MY_CHECKED_WRITE(fd, s) do { const auto
MY_CONCAT(_my_checked_write_, __LINE__) = write(fd, s.data(),
s.length()); MY_ASSERT_NOT_LESS_ZERO(MY_CONCAT(_my_checked_write_,
__LINE__)); MY_ASSERT(static_cast<size_t>(MY_CONCAT(_my_checked_write_,
__LINE__)) == s.length()); } while(0)

struct Xoroshiro128PP {
    uint64_t s[2];

    void seed(uint64_t x) {
        s[0] = splitmix64(x);
        s[1] = splitmix64(x);
    }

    static uint64_t splitmix64(uint64_t& state) {
        uint64_t z = (state += 0x9E3779B97F4A7C15);
        z = (z ^ (z >> 30)) * 0xBF58476D1CE4E5B9;
        z = (z ^ (z >> 27)) * 0x94D049BB133111EB;
        return z ^ (z >> 31);
    }

    static inline uint64_t rotl(uint64_t x, int k) {
        return (x << k) | (x >> (64 - k));
    }

    uint64_t next() {
        const uint64_t s0 = s[0];
```

```
        uint64_t s1 = s[1];
        const uint64_t result = rotl(s0 + s1, 17) + s0;
        s1 ^= s0;
        s[0] = rotl(s0, 49) ^ s1 ^ (s1 << 21);
        s[1] = rotl(s1, 28);
        return result;
    }

    double next_double() {
        return (next() >> 11) * (1.0 / 9007199254740992.0);
    }
};

int main() {
    Xoroshiro128PP xoroshiro{};
    xoroshiro.seed(static_cast<uint64_t>(time(NULL)));

    const auto current_file = std::filesystem::path(__FILE__);
    const auto project_dir = current_file.parent_path();
    const auto build_dir = project_dir / "build";
    const auto files_dir = build_dir / "test_dir";

    std::filesystem::remove_all(files_dir);
    MY_ASSERT_NOT_LESS_ZERO(mkdir(files_dir.c_str(), 0700));

    MY_FOR_RANGE_ZERO(file_index, 10000) {
        const auto file_path = files_dir / std::to_string(file_index);
        const int file_fd = open(file_path.c_str(), O_RDWR | O_CREAT |
O_TRUNC);
        MY_ASSERT_NOT_LESS_ZERO(file_fd);
        defer(MY_ASSERT_NOT_LESS_ZERO(close(file_fd)));
        MY_ASSERT_NOT_LESS_ZERO(chmod(file_path.c_str(), 0600));

        constexpr size_t words_count = 6000;
        constexpr size_t word_len = 50;
        constexpr size_t file_size = word_len * words_count;
        MY_ASSERT_NOT_LESS_ZERO(ftruncate(file_fd, file_size));
        struct stat st;
        MY_ASSERT_NOT_LESS_ZERO(fstat(file_fd, &st));
        MY_ASSERT(static_cast<size_t>(st.st_size) == file_size);

        char* const ptr = reinterpret_cast<char*>(mmap(nullptr,
static_cast<size_t>(st.st_size), PROT_WRITE | PROT_READ, MAP_SHARED,
file_fd, 0));
        MY_ASSERT(ptr != MAP_FAILED);
        defer(MY_ASSERT_NOT_LESS_ZERO(munmap(ptr,
static_cast<size_t>(st.st_size))));
```

```
        size_t index = 0;
        MY_FOR_RANGE_ZERO(word_index, words_count) {
            MY_FOR_RANGE_ZERO(letter_index, word_len - 1) {
                constexpr char rng_mod = 'z' - 'a';
                ptr[index++] = 'A' + (xoroshiro.next() % rng_mod);
            }
            ptr[index++] = '\n';
        }
    }
    return 0;
}
```

A.3 matrix_transpose.cpp

```
#include <concepts>
#include <cstdint>
#include <string>
#include <vector>
#include <array>
#include <chrono>
#include <thread>
#include <barrier>

#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <string.h>
#include <pthread.h>

#define MY_STRINGIFY_IMPL(...) #__VA_ARGS__
#define MY_S(...) MY_STRINGIFY_IMPL(__VA_ARGS__)
#define MY_CONCAT_INTERNAL(a, b) a ## b
#define MY_CONCAT(a, b) MY_CONCAT_INTERNAL(a, b)
#define MY_FOR_RANGE(type, name, mmin, mmax) for(type name = (mmin);
name < (mmax); name++)
#define MY_FOR_RANGE_ZERO(name, mmax) for(auto name =
static_cast<typename std::remove_cv<decltype(mmax)>::type>(0); name <
(mmax); name++)

#define MY_ASSERT(expr) \
    do {\
        if(!(expr)) {\
            fprintf(stderr, "%s:%d: %s: assertion `%s` failed.\n",
__FILE__, __LINE__, __FUNCTION__, #expr);\
            abort();\
        }\
    } while(0)
```

```
#define MY_ASSERT_EXT(expr, ...) \
    do {\
        if(!(expr)) {\
            fprintf(stderr, "%s:%d: %s: assertion `%s` failed.\n",
__FILE__, __LINE__, __FUNCTION__, #expr);\
            fprintf(stderr, __VA_ARGS__);\
            abort();\
        }\
    } while(0)

#define MY_ASSERT_NOT_LESS_ZERO(expr) \
    do {\
        if((expr) < 0) {\
            fprintf(stderr, "%s:%d: %s: errno: %d: strerror: %s:
assertion `%s' failed.\n", __FILE__, __LINE__, __FUNCTION__, errno,
strerror(errno), #expr);\
            abort();\
        }\
    } while(0)

#define MY_LOG_DEBUG_SINGLE(fmt) \
    do {\
        fprintf(stderr, "[%s:%d] [%s] [%ld] [" fmt "]\n", __FILE__,
__LINE__, __FUNCTION__, time(nullptr));\
    } while(0)

#define MY_LOG_DEBUG(fmt, ...) \
    do {\
        fprintf(stderr, "[%s:%d] [%s] [%ld] [" fmt "]\n", __FILE__,
__LINE__, __FUNCTION__, time(nullptr) __VA_OPT__(,) __VA_ARGS__);\
    } while(0)

#define MY_PRINT_EXPR_IMPL(expr, format)\
    do {\
        MY_LOG_DEBUG("`"#expr"`: `" format "`", (expr));\
    } while(0)


template <typename F>
struct defer_t {
    F f;
    ~defer_t() { f(); }
};
#define defer(code) const defer_t MY_CONCAT(_defer_, __LINE__){[&]()
{ code; }}

#define MY_ARRAY_COUNT(arr) (sizeof(arr) / sizeof(arr[0]))
```

```
#define MY_CHECKED_WRITE(fd, s) do { const auto
MY_CONCAT(_my_checked_write_, __LINE__) = write(fd, s.data(),
s.length()); MY_ASSERT_NOT_LESS_ZERO(MY_CONCAT(_my_checked_write_,
__LINE__)); MY_ASSERT(static_cast<size_t>(MY_CONCAT(_my_checked_write_,
__LINE__)) == s.length()); } while(0)

#define MY_PTHREAD_BARRIER_WAIT(barrier)\
    do {\
        switch(pthread_barrier_wait(barrier)) {\
            case 0:\
            case PTHREAD_BARRIER_SERIAL_THREAD: {\
                break;\
            }\
            default: {\
                MY_ASSERT(false);\
                break;\
            }\
        }\
    } while(0)

template<typename T>
static std::chrono::system_clock::duration timeit(T&& func) {
    const auto start = std::chrono::system_clock::now();
    func();
    return std::chrono::system_clock::now() - start;
}

static constexpr size_t cpus_count = 12;
static constexpr size_t samples_count = 10;

template<auto MMIN, auto MMAX, class F>
static constexpr void static_for(F&& f)
    requires std::same_as<decltype(MMIN), decltype(MMAX)> &&
std::integral<decltype(MMIN)>
{
    if constexpr (MMIN < MMAX) {
        std::forward<F>(f).template operator()<MMIN>();
        static_for<MMIN + 1, MMAX>(std::forward<F>(f));
    }
}

static void transpose(uint8_t matrix[], const size_t size) {
    MY_FOR_RANGE_ZERO(row, size - 1) {
        MY_FOR_RANGE(size_t, col, row + 1, size) {
            uint8_t temp;
            temp = matrix[row * size + col];
            matrix[row * size + col] = matrix[col * size + row];
            matrix[col * size + row] = temp;
```

```
        }
    }
}

static constexpr size_t BLOCK_SIZE = 32;

static void transpose_block_diag(uint8_t matrix[], const size_t
row_start, const size_t size) {
    const size_t limit = std::min(row_start + BLOCK_SIZE, size);
    for(size_t r = row_start; r < limit; ++r) {
        for(size_t c = r + 1; c < limit; ++c) {
            std::swap(matrix[r * size + c], matrix[c * size + r]);
        }
    }
}

static void transpose_block_swap(uint8_t* matrix, size_t r_start,
size_t c_start, size_t size) {
    const size_t r_limit = std::min(r_start + BLOCK_SIZE, size);
    const size_t c_limit = std::min(c_start + BLOCK_SIZE, size);
    for (size_t r = r_start; r < r_limit; ++r) {
        for (size_t c = c_start; c < c_limit; ++c) {
            std::swap(matrix[r * size + c], matrix[c * size + r]);
        }
    }
}

int main() {
    MY_ASSERT(sysconf(_SC_NPROCESSORS_ONLN) ==
static_cast<long>(cpus_count));

    const auto warm_up = []() {
        MY_LOG_DEBUG("start warm_up");
        const auto dur = timeit([&]() {
            constexpr uint64_t mmax =
std::numeric_limits<uint32_t>::max() >> 8;
            constexpr uint64_t mmin = mmax >> 1;
            std::array<std::jthread, cpus_count> threads{};
            for(auto& t : threads) {
                t = std::jthread([]() {
                    constexpr size_t matrix_size = 20000;
                    std::vector<uint8_t> matrix(matrix_size *
matrix_size);
                    transpose(matrix.data(), matrix_size);
                });
            }
        });
        const auto dur_casted =
```

```
std::chrono::duration_cast<std::chrono::duration<double,
std::milli>>(dur);
        MY_LOG_DEBUG("end warm_up, took %lf ms", dur_casted.count());
    };

    warm_up();
    static constexpr size_t matrix_size = 30000;

    {
        MY_LOG_DEBUG("start single");
        const auto fd = open("matrix_transpose_single.csv", O_RDWR |
O_CREAT | O_TRUNC);
        MY_ASSERT_NOT_LESS_ZERO(fd);
        defer(MY_ASSERT_NOT_LESS_ZERO(close(fd)));
        MY_ASSERT_NOT_LESS_ZERO(fchmod(fd, 0666));
        constexpr std::string_view header = "nanoseconds\n";
        MY_CHECKED_WRITE(fd, header);
        MY_FOR_RANGE_ZERO(sample_index, samples_count) {
            std::vector<uint8_t> matrix(matrix_size * matrix_size);
            const auto dur = timeit([&matrix]()
{ transpose(matrix.data(), matrix_size); });
            const auto line = std::to_string(dur.count()) + "\n";
            MY_CHECKED_WRITE(fd, line);
            printf("%s", line.data());
        }
        MY_LOG_DEBUG("end single");
    }
    warm_up();
    {
        MY_LOG_DEBUG("start multi");

        const int fd = open("matrix_transpose_multi.csv", O_RDWR |
O_CREAT | O_TRUNC);
        MY_ASSERT_NOT_LESS_ZERO(fd);
        defer(MY_ASSERT_NOT_LESS_ZERO(close(fd)));
        MY_ASSERT_NOT_LESS_ZERO(fchmod(fd, 0666));
        constexpr std::string_view header =
"threads_count,nanoseconds\n";
        MY_CHECKED_WRITE(fd, header);

        MY_FOR_RANGE(size_t, threads_count, 2, cpus_count + 1) {
            MY_FOR_RANGE_ZERO(sample_index, samples_count) {
                pthread_barrier_t barrier{};
                MY_ASSERT_NOT_LESS_ZERO(pthread_barrier_init(&barrier,
nullptr, static_cast<uint32_t>(threads_count + 1)));

defer(MY_ASSERT_NOT_LESS_ZERO(pthread_barrier_destroy(&barrier)));
```

```
                std::vector<uint8_t> matrix(matrix_size * matrix_size);
                std::vector<std::jthread> threads(threads_count);
                MY_FOR_RANGE_ZERO(thread_index, threads_count) {
                    threads[thread_index] = std::jthread(
                        [&barrier, thread_index, threads_count,
&matrix]() {
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                            for(size_t row = thread_index; row <
matrix_size - 1; row += threads_count) {
                                for(size_t col = row + 1; col <
matrix_size; ++col) {
                                    uint8_t temp = matrix[row *
matrix_size + col];
                                    matrix[row * matrix_size + col] =
matrix[col * matrix_size + row];
                                    matrix[col * matrix_size + row] =
temp;
                                }
                            }
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                        }
                    );
                }
                MY_PTHREAD_BARRIER_WAIT(&barrier);
                const auto dur = timeit(
                    [&barrier]() {
                        MY_PTHREAD_BARRIER_WAIT(&barrier);
                        MY_PTHREAD_BARRIER_WAIT(&barrier);
                    }
                );
                const auto line = std::to_string(threads_count) + "," +
std::to_string(dur.count()) + "\n";
                MY_CHECKED_WRITE(fd, line);
                printf("%s", line.data());
            }
        }
        MY_LOG_DEBUG("end multi");
    }
    warm_up();
    {
        MY_LOG_DEBUG("start multi");

        const int fd = open("matrix_transpose_block.csv", O_RDWR |
O_CREAT | O_TRUNC);
        MY_ASSERT_NOT_LESS_ZERO(fd);
        defer(MY_ASSERT_NOT_LESS_ZERO(close(fd)));
        MY_ASSERT_NOT_LESS_ZERO(fchmod(fd, 0666));
```

```
        constexpr std::string_view header =
"threads_count,nanoseconds\n";
        MY_CHECKED_WRITE(fd, header);

        MY_FOR_RANGE(size_t, threads_count, 2, cpus_count + 1) {
            MY_FOR_RANGE_ZERO(sample_index, samples_count) {
                pthread_barrier_t barrier{};
                MY_ASSERT_NOT_LESS_ZERO(pthread_barrier_init(&barrier,
nullptr, static_cast<uint32_t>(threads_count + 1)));
defer(MY_ASSERT_NOT_LESS_ZERO(pthread_barrier_destroy(&barrier)));

                std::vector<uint8_t> matrix(matrix_size * matrix_size);
                std::vector<std::jthread> threads(threads_count);
                MY_FOR_RANGE_ZERO(thread_index, threads_count) {
                    threads[thread_index] = std::jthread(
                        [&barrier, thread_index, threads_count,
&matrix]() {
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                            for(size_t i = thread_index * BLOCK_SIZE; i
< matrix_size; i += threads_count * BLOCK_SIZE) {
                                for(size_t j = i; j < matrix_size; j +=
BLOCK_SIZE) {
                                    if (i == j) {
transpose_block_diag(matrix.data(), i, matrix_size);
                                    } else {
transpose_block_swap(matrix.data(), i, j, matrix_size);
                                    }
                                }
                            }
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                        }
                    );
                }
                MY_PTHREAD_BARRIER_WAIT(&barrier);
                const auto dur = timeit(
                    [&barrier]() {
                        MY_PTHREAD_BARRIER_WAIT(&barrier);
                        MY_PTHREAD_BARRIER_WAIT(&barrier);
                    }
                );
                const auto line = std::to_string(threads_count) + "," +
std::to_string(dur.count()) + "\n";
                MY_CHECKED_WRITE(fd, line);
                printf("%s", line.data());
            }
        }
```

```
        MY_LOG_DEBUG("end multi");
    }
    return 0;
}
```

A.4 pi_monte.cpp

```
#include <algorithm>
#include <numeric>
#include <string>
#include <vector>
#include <array>
#include <chrono>
#include <thread>
#include <numbers>

#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <string.h>
#include <pthread.h>

#define MY_STRINGIFY_IMPL(...) #__VA_ARGS__
#define MY_S(...) MY_STRINGIFY_IMPL(__VA_ARGS__)
#define MY_CONCAT_INTERNAL(a, b) a ## b
#define MY_CONCAT(a, b) MY_CONCAT_INTERNAL(a, b)
#define MY_FOR_RANGE(type, name, mmin, mmax) for(type name = (mmin);
name < (mmax); name++)
#define MY_FOR_RANGE_ZERO(name, mmax) for(auto name =
static_cast<typename std::remove_cv<decltype(mmax)>::type>(0); name <
(mmax); name++)

#define MY_ASSERT(expr) \
    do {\
        if(!(expr)) {\
            fprintf(stderr, "%s:%d: %s: assertion `%s` failed.\n",
__FILE__, __LINE__, __FUNCTION__, #expr);\
            abort();\
        }\
    } while(0)

#define MY_ASSERT_EXT(expr, ...) \
    do {\
        if(!(expr)) {\
            fprintf(stderr, "%s:%d: %s: assertion `%s` failed.\n",
__FILE__, __LINE__, __FUNCTION__, #expr);\
            fprintf(stderr, __VA_ARGS__);\
            abort();\
```

```
        }\
    } while(0)

#define MY_ASSERT_NOT_LESS_ZERO(expr) \
    do {\
        if((expr) < 0) {\
            fprintf(stderr, "%s:%d: %s: errno: %d: strerror: %s:
assertion `%s' failed.\n", __FILE__, __LINE__, __FUNCTION__, errno,
strerror(errno), #expr);\
            abort();\
        }\
    } while(0)

#define MY_LOG_DEBUG_SINGLE(fmt) \
    do {\
        fprintf(stderr, "[%s:%d] [%s] [%ld] [" fmt "]\n", __FILE__,
__LINE__, __FUNCTION__, time(nullptr));\
    } while(0)

#define MY_LOG_DEBUG(fmt, ...) \
    do {\
        fprintf(stderr, "[%s:%d] [%s] [%ld] [" fmt "]\n", __FILE__,
__LINE__, __FUNCTION__, time(nullptr) __VA_OPT__(,) __VA_ARGS__);\
    } while(0)

#define MY_PRINT_EXPR_IMPL(expr, format)\
    do {\
        MY_LOG_DEBUG("`"#expr"`: `" format "`", (expr));\
    } while(0)


template <typename F>
struct defer_t {
    F f;
    ~defer_t() { f(); }
};

#define defer(code) const auto MY_CONCAT(_defer_lambda_, __LINE__) =
[&](){ code; }; defer_t<decltype(MY_CONCAT(_defer_lambda_, __LINE__))>
MY_CONCAT(_defer_, __LINE__){MY_CONCAT(_defer_lambda_, __LINE__)}

#define MY_ARRAY_COUNT(arr) (sizeof(arr) / sizeof(arr[0]))

#define MY_PTHREAD_BARRIER_WAIT(barrier)\
    do {\
        switch(pthread_barrier_wait(barrier)) {\
            case 0:\
            case PTHREAD_BARRIER_SERIAL_THREAD: {\
```

```
                break;\
            }\
            default: {\
                MY_ASSERT(false);\
                break;\
            }\
        }\
    } while(0)

template<typename T>
static std::chrono::system_clock::duration timeit(T&& func) {
    const auto start = std::chrono::system_clock::now();
    func();
    return std::chrono::system_clock::now() - start;
}

struct Xoroshiro128PP {
    uint64_t s[2];

    void seed(uint64_t x) {
        s[0] = splitmix64(x);
        s[1] = splitmix64(x);
    }

    static uint64_t splitmix64(uint64_t& state) {
        uint64_t z = (state += 0x9E3779B97F4A7C15);
        z = (z ^ (z >> 30)) * 0xBF58476D1CE4E5B9;
        z = (z ^ (z >> 27)) * 0x94D049BB133111EB;
        return z ^ (z >> 31);
    }

    static inline uint64_t rotl(uint64_t x, int k) {
        return (x << k) | (x >> (64 - k));
    }

    uint64_t next() {
        const uint64_t s0 = s[0];
        uint64_t s1 = s[1];
        const uint64_t result = rotl(s0 + s1, 17) + s0;
        s1 ^= s0;
        s[0] = rotl(s0, 49) ^ s1 ^ (s1 << 21);
        s[1] = rotl(s1, 28);
        return result;
    }

    double next_double() {
        return (next() >> 11) * (1.0 / 9007199254740992.0);
    }
```

```
};

static uint64_t count_points(const uint64_t points_count) {
    Xoroshiro128PP rng{};
    rng.seed(static_cast<uint64_t>(time(nullptr)));
    uint64_t points_inside_circle_count = 0;
    for(uint32_t i = 0; i < points_count; ++i) {
        const double x = rng.next_double() * 2.0 - 1.0;
        const double y = rng.next_double() * 2.0 - 1.0;
        if ((x * x + y * y) <= 1.0) {
            ++points_inside_circle_count;
        }
    }
    return points_inside_circle_count;
}

static constexpr size_t cpus_count = 12;
static constexpr size_t samples_count = 10;

int main() {
    MY_ASSERT(sysconf(_SC_NPROCESSORS_ONLN) ==
static_cast<long>(cpus_count));

    const auto warm_up = []() {
        MY_LOG_DEBUG("start warm_up");
        const auto dur = timeit([&]() {
            std::array<std::jthread, cpus_count> threads{};
            for(auto& thread : threads) {
                thread = std::jthread([]()
{ count_points(10'000'000); });
            }
        });
        const auto dur_casted =
std::chrono::duration_cast<std::chrono::duration<double,
std::milli>>(dur);
        MY_LOG_DEBUG("end warm_up, took %lf ms", dur_casted.count());
    };
    warm_up();

    const std::vector<uint64_t> points_counts{
        100'000'000,
        200'000'000,
        300'000'000,
        400'000'000,
        500'000'000,
        600'000'000,
        700'000'000,
        800'000'000,
```

```
        900'000'000,
        1'000'000'000,
    };
    {
        MY_LOG_DEBUG("start multi");
        const auto fd = open("pi_monte_single.csv", O_RDWR | O_CREAT |
O_TRUNC);
        MY_ASSERT_NOT_LESS_ZERO(fd);
        defer(MY_ASSERT_NOT_LESS_ZERO(close(fd)));
        MY_ASSERT_NOT_LESS_ZERO(fchmod(fd, 0666));
        constexpr std::string_view header =
"points_count,nanoseconds\n";
        MY_ASSERT_NOT_LESS_ZERO(write(fd, header.data(),
header.length()));
        for(const auto points_count : points_counts) {
            MY_FOR_RANGE_ZERO(sample_index, samples_count) {
                uint64_t points_circle_count = 0;
                const auto dur = timeit(
                    [points_count, &points_circle_count]() {
                        points_circle_count =
count_points(points_count);
                    });
                const double pi = 4.0 *
static_cast<double>(points_circle_count) /
static_cast<double>(points_count);
                MY_ASSERT(std::abs(pi - std::numbers::pi) < 0.01);

                const auto s = std::to_string(points_count) + "," +
std::to_string(dur.count()) + "\n";
                MY_ASSERT_NOT_LESS_ZERO(write(fd, s.c_str(),
s.length()));
                printf("%s", s.data());
            }
        }
        MY_LOG_DEBUG("end multi");
    }
    warm_up();
    {
        MY_LOG_DEBUG("start multi");
        const int fd = open("pi_monte_multi.csv", O_RDWR | O_CREAT |
O_TRUNC);
        MY_ASSERT_NOT_LESS_ZERO(fd);
        defer(MY_ASSERT_NOT_LESS_ZERO(close(fd)));
        MY_ASSERT_NOT_LESS_ZERO(fchmod(fd, 0666));
        constexpr std::string_view header =
"points_count,threads_count,nanoseconds\n";
        MY_ASSERT_NOT_LESS_ZERO(write(fd, header.data(),
header.length()));
```

```
        for(const auto points_count : points_counts) {
            for(uint32_t threads_count = 2; threads_count <=
cpus_count; ++threads_count) {

                pthread_barrier_t barrier{};
                MY_ASSERT_NOT_LESS_ZERO(pthread_barrier_init(&barrier,
nullptr, threads_count + 1));

defer(MY_ASSERT_NOT_LESS_ZERO(pthread_barrier_destroy(&barrier)));

                std::vector<std::jthread> threads(threads_count);
                std::vector<uint64_t>
points_circle_counts(threads_count);

                const auto range_len = points_count;
                MY_ASSERT(range_len > threads_count);
                const auto step = range_len / threads_count;
                const auto remainder = range_len % threads_count;
                for(size_t thread_index = 0; thread_index <
threads_count; ++thread_index) {
                    const auto points_per_thread = thread_index <
remainder ? step + 1 : step;
                    threads[thread_index] = std::jthread(
                        [&barrier, points_per_thread ,
&points_circle_counts, thread_index, threads_count]() {
                            MY_FOR_RANGE_ZERO(sample_index,
samples_count) {
                                MY_PTHREAD_BARRIER_WAIT(&barrier);
                                MY_PTHREAD_BARRIER_WAIT(&barrier);
                                points_circle_counts[thread_index] =
count_points(points_per_thread);
                                MY_PTHREAD_BARRIER_WAIT(&barrier);
                            }
                        }
                    );
                }

                MY_FOR_RANGE_ZERO(sample_index, samples_count) {
                    MY_PTHREAD_BARRIER_WAIT(&barrier);
                    const auto dur = timeit(
                        [&barrier ,&points_circle_counts, points_count,
threads_count]() {
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                            const double pi = 4.0 *
static_cast<double>(std::accumulate(std::begin(points_circle_counts),
std::end(points_circle_counts), 0)) /
```

```
static_cast<double>(points_count);
                            MY_ASSERT(std::abs(pi - std::numbers::pi) <
0.01);
                        }
                    );
static_assert(std::is_same<std::remove_cv_t<decltype(dur)>,
std::chrono::nanoseconds>::value, "");
                    const auto s = std::to_string(points_count) + "," +
std::to_string(threads_count) + "," + std::to_string(dur.count()) +
"\n";
                    MY_ASSERT_NOT_LESS_ZERO(write(fd, s.c_str(),
s.length()));
                    printf("%s", s.data());
                }
            }
            MY_LOG_DEBUG("end multi");
        }
    }
}
```

A.5 prime_slice.cpp

```
#include <cstdint>
#include <string>
#include <vector>
#include <array>
#include <chrono>
#include <thread>

#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <string.h>
#include <pthread.h>

#define MY_STRINGIFY_IMPL(...) #__VA_ARGS__
#define MY_S(...) MY_STRINGIFY_IMPL(__VA_ARGS__)
#define MY_CONCAT_INTERNAL(a, b) a ## b
#define MY_CONCAT(a, b) MY_CONCAT_INTERNAL(a, b)
#define MY_FOR_RANGE(type, name, mmin, mmax) for(type name = (mmin);
name < (mmax); name++)
#define MY_FOR_RANGE_ZERO(name, mmax) for(auto name =
static_cast<typename std::remove_cv<decltype(mmax)>::type>(0); name <
(mmax); name++)

#define MY_ASSERT(expr) \
    do {\
        if(!(expr)) {\
```

```
            fprintf(stderr, "%s:%d: %s: assertion `%s` failed.\n",
__FILE__, __LINE__, __FUNCTION__, #expr);\
            abort();\
        }\
    } while(0)

#define MY_ASSERT_EXT(expr, ...) \
    do {\
        if(!(expr)) {\
            fprintf(stderr, "%s:%d: %s: assertion `%s` failed.\n",
__FILE__, __LINE__, __FUNCTION__, #expr);\
            fprintf(stderr, __VA_ARGS__);\
            abort();\
        }\
    } while(0)

#define MY_ASSERT_NOT_LESS_ZERO(expr) \
    do {\
        if((expr) < 0) {\
            fprintf(stderr, "%s:%d: %s: errno: %d: strerror: %s:
assertion `%s' failed.\n", __FILE__, __LINE__, __FUNCTION__, errno,
strerror(errno), #expr);\
            abort();\
        }\
    } while(0)

#define MY_LOG_DEBUG_SINGLE(fmt) \
    do {\
        fprintf(stderr, "[%s:%d] [%s] [%ld] [" fmt "]\n", __FILE__,
__LINE__, __FUNCTION__, time(nullptr));\
    } while(0)

#define MY_LOG_DEBUG(fmt, ...) \
    do {\
        fprintf(stderr, "[%s:%d] [%s] [%ld] [" fmt "]\n", __FILE__,
__LINE__, __FUNCTION__, time(nullptr) __VA_OPT__(,) __VA_ARGS__);\
    } while(0)

#define MY_PRINT_EXPR_IMPL(expr, format)\
    do {\
        MY_LOG_DEBUG("`"#expr"`: `" format "`", (expr));\
    } while(0)


template <typename F>
struct defer_t {
    F f;
    ~defer_t() { f(); }
```

```
};
#define defer(code) const defer_t MY_CONCAT(_defer_, __LINE__){[&]()
{ code; }}

#define MY_ARRAY_COUNT(arr) (sizeof(arr) / sizeof(arr[0]))

#define MY_PTHREAD_BARRIER_WAIT(barrier)\
    do {\
        switch(pthread_barrier_wait(barrier)) {\
            case 0:\
            case PTHREAD_BARRIER_SERIAL_THREAD: {\
                break;\
            }\
            default: {\
                MY_ASSERT(false);\
                break;\
            }\
        }\
    } while(0)

template<typename T>
static std::chrono::system_clock::duration timeit(T&& func) {
    const auto start = std::chrono::system_clock::now();
    func();
    return std::chrono::system_clock::now() - start;
}

static bool is_prime(uint64_t n) {
    if(n % 2 == 0) {
        return true;
    }
    for(uint64_t d = 3; d * d <= n; d += 2) {
        if(n % d == 0) {
            return true;
        }
    }
    return false;
}

static constexpr size_t cpus_count = 12;
static constexpr size_t samples_count = 10;

#define MY_CHECKED_WRITE(fd, s) do { const auto
MY_CONCAT(_my_checked_write_, __LINE__) = write(fd, s.data(),
s.length()); MY_ASSERT_NOT_LESS_ZERO(MY_CONCAT(_my_checked_write_,
__LINE__)); MY_ASSERT(static_cast<size_t>(MY_CONCAT(_my_checked_write_,
__LINE__)) == s.length()); } while(0)
```

```
int main() {
    MY_ASSERT(sysconf(_SC_NPROCESSORS_ONLN) ==
static_cast<long>(cpus_count));

    const int dev_null_fd = open("/dev/null", O_WRONLY);
    MY_ASSERT_NOT_LESS_ZERO(dev_null_fd);
    defer(MY_ASSERT_NOT_LESS_ZERO(close(dev_null_fd)));

    const auto warm_up = [dev_null_fd]() {
        MY_LOG_DEBUG("start warm_up");
        const auto dur = timeit([&]() {
            constexpr uint64_t mmax =
std::numeric_limits<uint32_t>::max() >> 8;
            constexpr uint64_t mmin = mmax >> 1;
            std::array<std::jthread, cpus_count> threads{};
            for(auto& t : threads) {
                t = std::jthread([dev_null_fd]() {
                    MY_FOR_RANGE(uint64_t, i, mmin, mmax) {
                        const bool res = is_prime(i);
                        write(dev_null_fd, &res, sizeof(res));
                    }
                });
            }
        });
        const auto dur_casted =
std::chrono::duration_cast<std::chrono::duration<double,
std::milli>>(dur);
        MY_LOG_DEBUG("end warm_up, took %lf ms", dur_casted.count());
    };

    warm_up();

    MY_LOG_DEBUG("start single");
    {
        const auto fd = open("prime_slice_single.csv", O_RDWR | O_CREAT
| O_TRUNC);
        MY_ASSERT_NOT_LESS_ZERO(fd);
        defer(MY_ASSERT_NOT_LESS_ZERO(close(fd)));
        MY_ASSERT_NOT_LESS_ZERO(fchmod(fd, 0666));
        constexpr std::string_view header = "mmin,mmax,nanoseconds\n";
        MY_CHECKED_WRITE(fd, header);

        MY_FOR_RANGE(uint64_t, i, 7, 10) {
            const uint64_t mmax = std::numeric_limits<uint32_t>::max()
>> i;
            const uint64_t mmin = mmax >> 1;

            MY_FOR_RANGE_ZERO(sample_index, samples_count) {
```

```
                const auto dur = timeit([dev_null_fd, mmin, mmax]() {
                    MY_FOR_RANGE(uint64_t, number, mmin, mmax) {
                        const bool res = is_prime(number);
                        write(dev_null_fd, &res, sizeof(res));
                    }
                });
                const auto line = std::to_string(mmin) + "," +
std::to_string(mmax) + "," + std::to_string(dur.count()) + "\n";
                MY_CHECKED_WRITE(fd, line);
                printf("%s", line.data());
            }
        }
    }
    MY_LOG_DEBUG("end single");

    warm_up();

    MY_LOG_DEBUG("start multi");
    {
        const int fd = open("prime_slice_multi.csv", O_RDWR | O_CREAT |
O_TRUNC);
        MY_ASSERT_NOT_LESS_ZERO(fd);
        defer(MY_ASSERT_NOT_LESS_ZERO(close(fd)));
        MY_ASSERT_NOT_LESS_ZERO(fchmod(fd, 0666));
        constexpr std::string_view header =
"threads_count,mmin,mmax,nanoseconds\n";
        MY_CHECKED_WRITE(fd, header);

        MY_FOR_RANGE(uint64_t, i, 7, 10) {
            const uint64_t mmax = std::numeric_limits<uint32_t>::max()
>> i;
            const uint64_t mmin = mmax >> 1;
            MY_FOR_RANGE(size_t, threads_count, 2, cpus_count) {
                pthread_barrier_t barrier{};
                MY_ASSERT_NOT_LESS_ZERO(pthread_barrier_init(&barrier,
nullptr, static_cast<uint32_t>(threads_count + 1)));
defer(MY_ASSERT_NOT_LESS_ZERO(pthread_barrier_destroy(&barrier)));

                std::vector<std::jthread> threads(threads_count);

                const auto range_len = mmax - mmin;
                const auto step = range_len / threads_count;
                const auto remainder = range_len % threads_count;
                for(size_t thread_index = 0, offset = 0; thread_index <
threads_count; ++thread_index) {
                    uint64_t local_mmin = 0;
                    uint64_t local_mmax = 0;
```

```
                    if(offset >= range_len) {
                        local_mmin = mmax;
                        local_mmax = mmax;
                    } else {
                        const auto local_step = thread_index <
remainder ? step + 1 : step;
                        local_mmin = mmin + offset;
                        local_mmax = mmin + offset + local_step;
                        offset += local_step;
                    }
                    threads[thread_index] = std::jthread(
                        [&barrier, dev_null_fd, local_mmin, local_mmax]
() {
                            MY_FOR_RANGE_ZERO(sample_index,
samples_count) {
                                MY_PTHREAD_BARRIER_WAIT(&barrier);
                                MY_PTHREAD_BARRIER_WAIT(&barrier);

                                MY_FOR_RANGE(uint64_t, number,
local_mmin, local_mmax) {
                                    const bool res = is_prime(number);
                                    write(dev_null_fd, &res,
sizeof(res));
                                }

                                MY_PTHREAD_BARRIER_WAIT(&barrier);
                            }
                        }
                    );
                }

                MY_FOR_RANGE_ZERO(sample_index, samples_count) {
                    MY_PTHREAD_BARRIER_WAIT(&barrier);
                    const auto dur = timeit(
                        [&barrier]() {
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                            MY_PTHREAD_BARRIER_WAIT(&barrier);
                        }
                    );
                    const auto line = std::to_string(threads_count) +
"," + std::to_string(mmin) + "," + std::to_string(mmax) + "," +
std::to_string(dur.count()) + "\n";
                    MY_CHECKED_WRITE(fd, line);
                    printf("%s", line.data());
                }
            }
        }
    }
```

```
    MY_LOG_DEBUG("end multi");
    return 0;
}
```